Repository Notes: EGMN (Watch Time Prediction)

1 What is in this repository?

This repository is a PyTorch research codebase for **video watch-time prediction**. It contains an implementation of the paper:

> *"Multi-Granularity Distribution Modeling for Video Watch Time Prediction via Exponential-Gaussian Mixture Network"*

The repository also includes several baseline models and code to preprocess the public KuaiRec dataset.

1.1 High-level structure

- `model/`: Model implementations (proposed EGMN and baselines).
- `dataloader/`: Dataset loaders that read preprocessed files and provide PyTorch `DataLoaders`.
- `dataset/kuairec/`: KuaiRec preprocessing scripts.
- `run_*.py`: Entrypoints to train/evaluate each method.
- `utils.py`: Evaluation metrics (MAE, XAUC, KL divergence) and helper utilities.

1.2 Code examples

Typical entrypoint invocation

```
# Train + evaluate the proposed EGMN model on KuaiRec
python run_egmn.py --dataset_name kuairec --device cuda:0

# Baselines (examples)
python run_cread.py --dataset_name kuairec
python run_d2q.py --dataset_name kuairec
```

What the dataloader yields The KuaiRec dataloader returns a tuple `(features_dict, label)`, where `features_dict` contains tensors keyed by feature name (e.g. `duration`, categorical IDs, and sequence masks).

```
# Pseudocode reflecting dataloader/kuairec.py
for features, label in dataloaders["train"]:
    # features is a dict of tensors, label is the normalized play_time
    y_hat = model.predict(features)
```

Metrics used in evaluation

```
# utils.py (simplified)
def eval_mae(labels, scores):
    return np.mean(np.abs(labels - scores))

def eval_xauc(labels, preds):
    # ranks by prediction, then measures label ordering agreement
    ...

def eval_kl(samples_p, samples_q, bins=100, epsilon=1e-10):
    hist_p, bin_edges = np.histogram(samples_p, bins=bins, density=True)
    hist_q, _ = np.histogram(samples_q, bins=bin_edges, density=True)
    ...
    return np.sum(hist_p * np.log(hist_p / hist_q))
```

1.3 Evaluation outputs

The run scripts train on `train` and evaluate on `test`, printing (at least):

- **MAE**: mean absolute error on watch-time prediction.
- **XAUC**: a ranking-style metric computed from label–prediction ordering.
- **KL**: KL divergence between binned empirical distributions.

2 What is the model (EGMN)?

The main proposed model in this repository is **EGMN** (Exponential–Gaussian Mixture Network), implemented in `model/egmn.py`. EGMN models watch time as a **mixture distribution** designed to capture both short "quick-skip" behavior and longer watch-time behavior.

2.1 Distribution family

EGMN uses a mixture with:

- **One Exponential component** (to capture the spike near zero watch time).
- **Multiple Gaussian components** (to capture longer watch time). In the code, the Gaussian components are implemented as **Normals truncated at 0** (to avoid negative time).

If we denote the mixture weights by π, exponential rate by λ, and Gaussian parameters (μ_k, σ_k), the model defines a mixture density over watch time $y \geq 0$ of the form:

$$p(y) = \pi_0 \,\mathrm{Exp}(y; \lambda) + \sum_{k=1}^{K} \pi_k \,\mathrm{TN}(y; \mu_k, \sigma_k, \mathrm{lower} = 0),$$

where $\mathrm{TN}(\cdot)$ is a Normal distribution truncated below at 0.

2.2 Neural architecture

- **Inputs**: a feature dictionary produced by the dataloader (sparse IDs, sequence features with masks, and continuous features such as `duration`).
- **Feature encoding**: embeddings for sparse/sequence features, and linear transforms for continuous features.
- **Shared trunk**: a multi-layer perceptron (MLP) over the concatenated feature representation.
- **Output heads**: predict mixture logits (converted to weights by softmax), the exponential rate λ (via softplus), and Gaussian parameters μ, σ (also via softplus for positivity constraints in the implementation).

2.3 Code snippet: heads and parameterization

```
# model/egmn.py (illustrative excerpt)
self.lambda_layer = nn.Sequential(
    nn.Linear(hidden_dim, 1),
    nn.Softplus(beta=0.5),
)

comp_num = 10 # number of Gaussian components
self.mixture_logits = nn.Linear(hidden_dim, comp_num + 1) # +1 for Exponential
self.gauss_mu = nn.Sequential(nn.Linear(hidden_dim, comp_num), nn.Softplus())
self.gauss_sigma = nn.Sequential(nn.Linear(hidden_dim, comp_num), nn.Softplus())
```

2.4 Training objective (as implemented)

Training uses a combination of:

- **Negative log-likelihood (NLL)** of the observed watch time under the mixture.
- **Reconstruction regularizer**: an L1 loss between the observed label y and the model's mixture *mean* prediction.
- **Mixture entropy term**: computed from the mixture weights π, weighted by a hyperparameter (`--alpha` in `run_egmn.py`).

In the training script, the total loss is:

$$\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha\, \mathcal{L}_{\text{entropy}} + \beta\, \mathcal{L}_{\text{reg}},$$

with α and β controlled by command-line flags.

2.5 Prediction used for evaluation

For evaluation, the code uses a **point prediction equal to the mixture mean**. In the implementation, the predicted value is computed as a weighted sum of component means (including the exponential mean $1/\lambda$).

2.6 Code snippet: prediction as mixture mean

```
# model/egmn.py (illustrative excerpt)
pi, lambda_, mu, sigma = self.forward(features)
pi = torch.softmax(pi, dim=1)

# component means: Exponential mean is 1/lambda; Gaussians use mu
component_means = torch.concat([1 / lambda_, mu], dim=1)
pred = torch.sum(pi * component_means, dim=1)
```

3 Is this a retrieval model or a ranking model? What are the outputs?

3.1 Short answer

This repository implements **watch-time prediction** for already-observed (user, item/video) interactions. In that sense, the provided training/evaluation scripts operate as a **pointwise regression / distribution modeling** setup rather than a classic **retrieval** system.

3.2 Retrieval vs. ranking vs. pointwise prediction

- **Retrieval** typically means: given a query/user context, score (often via approximate nearest neighbors) a very large candidate set to retrieve top-K items.
- **Ranking** typically means: given a smaller candidate set, assign scores and sort to produce an ordered list.
- **Pointwise prediction** (what the run scripts do here) means: predict a target value for each example independently (e.g. watch time) and optimize a pointwise loss (NLL, MAE/L1/MSE-like components).

In this repo, the scripts in `run_*.py` iterate over a `DataLoader` of interaction records and predict a watch-time value/distribution for each record. They do not generate candidates, do not compute top-K lists, and do not train with a pairwise/listwise ranking loss.

3.3 What does the model output?

There are two useful notions of "output":

- **Internal outputs** used to define a distribution / loss (e.g. mixture parameters).
- **Final prediction** used for evaluation (a scalar watch-time prediction).

3.4 EGMN outputs

EGMN (`model/egmn.py`) produces parameters of a mixture distribution:

- mixture logits π (converted to mixture weights via softmax),
- exponential rate λ,

- Gaussian parameters μ and σ for multiple components.

The evaluation code uses a **scalar point prediction** computed as the **mixture mean**.

```
# run_egmn.py (illustrative structure)
with torch.no_grad():
    for features, label in dataloaders["test"]:
        y = model.predict(features) # scalar per example
        # collect y for MAE / XAUC / KL

# model/egmn.py (illustrative)
pi, lambda_, mu, sigma = self.forward(features)
pi = torch.softmax(pi, dim=1)
component_means = torch.concat([1 / lambda_, mu], dim=1)
pred = torch.sum(pi * component_means, dim=1)
```

3.5 Baseline outputs (examples)

Different baselines output different intermediate objects, but they all ultimately provide a scalar prediction for evaluation:

Wide & Deep (`WideAndDeep`) Outputs a **single probability-like scalar** (sigmoid of an MLP + optional linear part). In this repo, it is used as a pointwise predictor.

CREAD (`Cread`) Outputs a **vector of bucket/head probabilities**. The training script discretizes the label into multiple binary thresholds and trains with BCE. At test time it reconstructs a **scalar watch-time estimate** from the bucket outputs.

D2Q (`D2Q`) Outputs a **normalized quantile score** (sigmoid). The training script maps labels into bucket-specific quantiles, trains with MSE, then maps predicted quantiles back to values.

```
# Examples of different "output shapes"

# Wide&Deep: scalar
p = wide_and_deep(features) # shape: [batch]

# CREAD: vector over M bucket heads
bucket_probs = cread(features) # shape: [batch, M]

# D2Q: scalar quantile (later mapped back to value)
q = d2q(features) # shape: [batch]
```

3.6 Is XAUC a ranking metric?

The repo reports **XAUC**, which is computed by sorting examples by predicted score and measuring agreement with label ordering. This is a **ranking-style evaluation metric**, but it does not mean the models are trained with a retrieval/ranking objective; training is still pointwise (NLL/regression-style) in the provided scripts.

4 What does gradient descent update in this model?

This section explains (at a practical implementation level) what parameters are updated by gradient-based optimization when training the models in this repository, focusing on the proposed **EGMN**.

4.1 What parameters are learnable?

In PyTorch, **gradient descent updates all tensors registered as parameters** (i.e. returned by `model.parameters()`). Concretely for EGMN, these include:

- **Embedding tables** for sparse and sequence features.
- **Linear layers** for continuous features (e.g. `duration` projection).
- **Shared MLP (trunk)** weights and biases.
- **Output heads** predicting the mixture parameters:
  - mixture logits π (pre-softmax),
  - exponential rate λ (via softplus),
  - Gaussian parameters μ and σ (via softplus in the implementation).

4.2 What is the objective being minimized?

During training, gradients are computed for a scalar loss. For EGMN, the training script forms:

$$\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha\, \mathcal{L}_{\text{entropy}} + \beta\, \mathcal{L}_{\text{reg}}.$$

- $\mathcal{L}_{\text{NLL}}$: negative log-likelihood of the observed watch time under the mixture distribution.
- $\mathcal{L}_{\text{reg}}$: reconstruction-style regularizer (L1 loss) between the observed label and the mixture-mean prediction.
- $\mathcal{L}_{\text{entropy}}$: a mixture entropy term computed from the mixture weights (controls component usage), weighted by α.

4.3 What does "gradient descent" do here?

At each iteration, PyTorch computes gradients $\nabla_\theta \mathcal{L}$ for all learnable parameters θ. The optimizer (e.g. Adagrad/Adam) then applies an update of the form:

$$\theta \leftarrow \theta - \eta\, \Delta(\nabla_\theta \mathcal{L}),$$

where η is the learning rate and $\Delta(\cdot)$ depends on the optimizer (e.g. per-parameter adaptive scaling).

4.4 Code example: the update step in training

The training scripts in this repo follow the standard PyTorch pattern:

```
# run_egmn.py / run_*.py (illustrative)
optimizer = torch.optim.Adagrad(model.parameters(), lr=args.lr,
                                weight_decay=args.weight_decay)

for features, label in dataloaders["train"]:
    # Forward: compute distribution parameters
    pi, lambda_, mu, sigma = model(features)

    # Loss: compute scalar objective
    nll, reg, ent = model.loss(label.float(), pi, lambda_, mu, sigma,
                               features["duration"].view(-1, 1))
    loss = nll + args.alpha * ent + args.beta * reg

    # Backward: accumulate gradients d(loss)/d(theta)
    optimizer.zero_grad()
    loss.backward()

    # Update: apply optimizer rule to parameters theta
    optimizer.step()
```

4.5 How gradients flow to mixture parameters

Even though EGMN conceptually outputs π, λ, μ, σ, these are *not* parameters themselves; they are **functions of the network weights**. Gradients flow from the loss through these outputs back into the head layers and the shared trunk.

```
# model/egmn.py (illustrative)
# pi is produced by a linear layer and normalized via softmax
+pi = self.mixture_logits(hidden) # logits
+mix_probs = torch.softmax(pi, dim=1) # weights

# lambda, mu, sigma are produced by heads with softplus constraints
+lambda_ = self.lambda_layer(hidden) + 1e-6
+mu = self.gauss_mu(hidden) + 1 / lambda_
+sigma = self.gauss_sigma(hidden) + 1e-6

# The NLL depends on (mix_probs, lambda_, mu, sigma),
# so gradients propagate into all head/trunk parameters.
```

4.6 Which parts are (not) updated?

- Updated: any parameters owned by the model modules passed to the optimizer.
- Not updated: input tensors (features/labels), data preprocessing artifacts (e.g. bucket boundaries / quantile tables), and any values created under `torch.no_grad()`.

5 Workflow: dataset preprocessing and running experiments

5.1 Dataset preprocessing (KuaiRec)

The KuaiRec raw data must be preprocessed before training. The provided script in `dataset/kuairec/kuairec_process.py`:

- merges multiple KuaiRec feature CSVs into a single table,
- builds vocabularies for sparse/sequence features,
- constructs a feature `description` schema used by models and dataloaders,
- splits into `train`/`test`,
- writes `kuairec_data.pkl` containing the processed data and schema.

It also produces duration buckets and per-bucket play-time quantiles used by the D2Q baseline.

5.2 Code snippet: preprocessing outputs (D2Q support)

```
# dataset/kuairec/kuairec_process.py (illustrative excerpt)
n_bins = 50
df["duration_bucket"], bins = pd.qcut(df["duration"], q=n_bins, labels=False,
                                      retbins=True, duplicates="drop")
desc.append(("duration_bucket", len(bins) - 1, "spr"))

# Per-bucket play-time quantiles for D2Q mapping
quantile_num = 100
quantiles = np.linspace(0, 1, quantile_num + 1)
quantile_df = (df_train.groupby("duration_bucket")["play_time"]
              .quantile(quantiles).reset_index())
quantile_df.pivot(index="duration_bucket", columns="quantile",
                  values="play_time").to_csv(
    "d2q_duration_bucket_playtime_quantiles.csv"
)
```

5.3 Training / evaluation entrypoints

Each method has a `run_*.py` entrypoint. Typical usage is:

- `python run_egmn.py --dataset_name kuairec`
- `python run_cread.py --dataset_name kuairec`
- `python run_d2q.py --dataset_name kuairec`

These scripts:

- load `./dataset/kuairec/kuairec_data.pkl` through `dataloader/kuairec.py`,
- train for a fixed number of epochs,
- evaluate on the test set,
- print MAE, XAUC, and KL divergence.

5.4 Code snippet: EGMN training loop structure

```
# run_egmn.py (illustrative excerpt)
for epoch in range(1, args.epoch + 1):
    for features, label in dataloaders["train"]:
        pi, lambda_, mu, sigma = model(features)
        nll, reg, ent = model.loss(label.float(), pi, lambda_, mu, sigma,
                                   features["duration"].view(-1, 1))
        loss = nll + args.alpha * ent + args.beta * reg
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

5.5 Dependencies

The README indicates the project was tested with Python 3.x and PyTorch (e.g. 2.1.0), and expects common scientific Python packages (NumPy, pandas, scikit-learn).